# Avaliação Teórica e Experimental dos Algoritmos de Ordenação MergeSort e TimSort

**João Vitor Santana Lima e Samuel Torres Vieira da Silva**

[1]Departamento de Sistemas de Informação – Universidade Federal do Piauí (UFPI)
64.600-000 – Picos – PI – Brasil

{joao.vitor.1, samuel.vieira}@ufpi.edu.br

***Resumo.*** *Este trabalho apresenta uma avaliação teórica e experimental dos algoritmos de ordenação Merge Sort e Tim Sort, analisando seu desempenho em diferentes cenários de entrada, incluindo dados crescentes, decrescentes e aleatórios. A investigação combina análise assintótica e experimentação empírica, permitindo relacionar previsões teóricas com resultados observados na prática. Os experimentos foram conduzidos utilizando vetores de diferentes tamanhos, executados repetidamente para obtenção de médias representativas, assegurando confiabilidade estatística nas medições. Os resultados demonstram que ambos os algoritmos mantêm complexidade O(n log n), porém apresentam diferenças relevantes nos coeficientes práticos de execução, evidenciando que o desempenho depende fortemente da estrutura inicial dos dados. Conclui-se que a escolha do algoritmo deve considerar o contexto de aplicação e as características da entrada, reforçando a importância da análise experimental como complemento à análise teórica em estudos de algoritmos.*

## 1. Introdução

A ordenação de dados constitui uma das operações mais fundamentais da ciência da computação,sendo amplamente estudada desde os primórdios da área e considerada essencial em diversas aplicações computacionais modernas [Knuth 1998, Aho et al. 1974, Weiss 2014]. Algoritmos como *Insertion Sort*, *Merge Sort*, *Heap Sort* e *Quick Sort* possuem complexidades e comportamentos distintos, cujo entendimento formal é indispensável para a escolha adequada em cada contexto [Cormen et al. 2009]. A análise teórica desses algoritmos envolve o estudo de complexidade de tempo e espaço, estabilidade e comportamento nos casos melhor, médio e pior [Sedgewick and Wayne 2011].

Embora a análise assintótica forneça limites formais importantes, estudos experimentais demonstram que o comportamento prático pode divergir das previsões teóricas em função da distribuição dos dados e das características do hardware [Sedgewick and Wayne 2011]. A validação empírica por meio de benchmarks com diferentes tamanhos e tipos de entrada complementa a análise formal, permitindo conclusões mais robustas sobre eficiência e escalabilidade [Cormen et al. 2009]. Essa abordagem experimental é amplamente adotada em estudos de desempenho de algoritmos, permitindo observar diferenças práticas que não são capturadas apenas pela análise assintótica [Bentley and McIlroy 1993, McIlroy 1993, Gonnet and Baeza-Yates 1991].

O *MergeSort*, proposto por John von Neumann em 1945, é um algoritmo baseado na estratégia de divisão e conquista que garante complexidade de tempo $O(n \log n)$ nos casos médio e pior, além de ser estável por natureza [Knuth 1998]. Sua estrutura recursiva divide

o conjunto de dados em subconjuntos menores, ordena-os individualmente e os combina por meio de fusões sucessivas, resultando em desempenho previsível independentemente da distribuição da entrada [Cormen et al. 2009]. *O Tim Sort*, por sua vez, é um algoritmo híbrido desenvolvido por Tim Peters em 2002, que combina *Merge Sort* e *Insertion Sort* para explorar subsequências já ordenadas, denominadas *runs*, presentes em dados do mundo real [Sedgewick and Wayne 2011].

A análise experimental de algoritmos de ordenação envolve a mensuração de métricas como tempo de execução, número de comparações, uso de memória e comportamento frente a diferentes distribuições de entrada [Cormen et al. 2009]. No caso do *Merge Sort*, os tempos de execução se mantêm consistentes entre os casos melhor, médio e pior, dado seu comportamento $O(n \log n)$ garantido, ao custo de uso de memória auxiliar proporcional a $n$ [Knuth 1998]. O *Tim Sort*, por ser adaptativo, apresenta desempenho superior em dados parcialmente ordenados, com complexidade $O(n)$ no melhor caso, superando o *Merge Sort* clássico em cenários práticos com entradas mistas [Wiredu et al. 2025].

Diante do exposto, o objetivo deste trabalho é realizar uma avaliação comparativa dos algoritmos de ordenação *Merge Sort* e *Tim Sort*, observando o desempenho, complexidade, assintótica e sensibilidade à ordenação inicial dos dados. Fundamentado com experimentos controlados, buscando relacionar os resultados experienciados com as previsões teóricas, contribuindo para uma compreensão mais sólida sobre a aplicação desses algoritmos em diferentes cenários.

## 2. Referencial Teórico

Nesta seção, serão mostrados os fundamentos teóricos relacionados aos algoritmos de ordenação analisados neste trabalho, tratando de suas características estruturais, seus princípios de funcionamento e particularidades operacionais. Serão tratados os aspectos conceituais que norteiam o comportamento dos algoritmos *Merge Sort* e *Tim Sort*, bem como suas estratégias internas para ordenação e impactos sobre o desempenho. Ademais, serão analisados as suas complexidades assintóticas, considerando diferentes tipos de entrada de dados.

### 2.1. Merge Sort

O algoritmo *Merge Sort* foi proposto por John von Neumann em 1945, sendo um dos primeiros métodos de ordenação concebidos para computadores eletrônicos [Knuth 1998]. Uma descrição detalhada e análise do algoritmo *bottom-up* foi publicada por Goldstine e von Neumann já em 1948, consolidando o *Merge Sort* como um dos algoritmos mais estudados na literatura de ciência da computação [Cormen et al. 2009].

O princípio fundamental do *Merge Sort* baseia-se no paradigma *Dividir e Conquistar* (*Divide and Conquer*), no qual o problema original é recursivamente decomposto em subproblemas menores, que são resolvidos individualmente e depois combinados para produzir a solução final [Cormen et al. 2009]. De forma concreta, o algoritmo divide o vetor de entrada em duas metades, ordena cada metade recursivamente por meio de chamadas ao próprio *Merge Sort* e, por fim, intercala (*merge*) as duas metades já ordenadas para produzir o vetor completamente ordenado [Sedgewick and Wayne 2011].

A etapa de intercalação é o núcleo operacional do algoritmo. Dados dois subvetores ordenados de tamanhos $n_1$ e $n_2$, o procedimento de *merge* percorre ambos simultaneamente,

comparando os elementos frontais e copiando o menor deles para um vetor auxiliar, até que todos os elementos sejam consumidos. Essa operação é executada em tempo linear $O(n_1 + n_2)$, o que garante a eficiência global do algoritmo [Cormen et al. 2009].

Do ponto de vista da complexidade assintótica, a recorrência que descreve o comportamento do *Merge Sort* é dada por:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \tag{1}$$

cuja solução, obtida pelo Teorema Mestre, resulta em $T(n) = \Theta(n \log n)$ [Cormen et al. 2009]. Essa complexidade se mantém nos casos médio, melhor e pior, tornando o *Merge Sort* um algoritmo de desempenho previsível e consistente independentemente da distribuição inicial dos dados [Knuth 1998, Sedgewick and Wayne 2011].

Em termos de complexidade de espaço, o *Merge Sort* convencional requer memória auxiliar de ordem $\Theta(n)$, utilizada durante a etapa de intercalação [Cormen et al. 2009]. Essa característica o classifica como um algoritmo *fora do lugar* (*not in-place*) [Sedgewick and Wayne 2011].

Entre suas propriedades operacionais, destaca-se o fato de o *Merge Sort* ser um algoritmo **estável**, preservando a ordem relativa de elementos com valores iguais presentes na entrada [Knuth 1998]. Essa estabilidade o torna especialmente adequado para aplicações que exigem múltiplos critérios de ordenação ou que dependem da preservação de ordem pré-existente nos dados [Sedgewick and Wayne 2011]. Além disso, o algoritmo apresenta desempenho naturalmente paralelizável, uma vez que as subpartes do vetor podem ser ordenadas de forma independente, característica explorada em arquiteturas multiprocessadas e em sistemas de ordenação externa [Cormen et al. 2009].

Por fim, o *Merge Sort* estabelece uma relação direta entre teoria e prática: sua complexidade teórica $\Theta(n \log n)$ é assintoticamente ótima para algoritmos de ordenação baseados em comparações, conforme demonstrado pelo limite inferior $\Omega(n \log n)$ para essa classe de algoritmos [Cormen et al. 2009, Knuth 1998]. A Listagem 1 apresenta uma implementação em pseudocódigo do algoritmo.

**Listing 1. Implementação em Pseudocódigo do Merge Sort**

```
PROCEDIMENTO mergeSort(v[], esq, dir)
  SE esq < dir ENTAO
    meio = (esq + dir) / 2
    mergeSort(v, esq, meio)       // Ordena metade esquerda
    mergeSort(v, meio + 1, dir)   // Ordena metade direita
    merge(v, esq, meio, dir)      // Intercala as duas metades
  FIM SE
FIM PROCEDIMENTO

PROCEDIMENTO merge(v[], esq, meio, dir)
  n1 = meio - esq + 1
  n2 = dir - meio
  CRIA vetores auxiliares E[n1] e D[n2]

  PARA i = 0 ATE n1 - 1 FACA
    E[i] = v[esq + i]
  FIM PARA
```

```
  PARA j = 0 ATE n2 - 1 FACA
    D[j] = v[meio + 1 + j]
  FIM PARA

  i = 0; j = 0; k = esq
  ENQUANTO i < n1 E j < n2 FACA
    SE E[i] <= D[j] ENTAO
      v[k] = E[i]; i = i + 1
    SENAO
      v[k] = D[j]; j = j + 1
    FIM SE
    k = k + 1
  FIM ENQUANTO

  ENQUANTO i < n1 FACA
    v[k] = E[i]; i = i + 1; k = k + 1
  FIM ENQUANTO

  ENQUANTO j < n2 FACA
    v[k] = D[j]; j = j + 1; k = k + 1
  FIM ENQUANTO
FIM PROCEDIMENTO
```

O parindent atual e: 36.13512pt

### 2.2. Tim Sort

O *Tim Sort* é um algoritmo de ordenação híbrido e estável, desenvolvido por Tim Peters em 2002 para ser utilizado como algoritmo padrão de ordenação da linguagem Python [Peters 2002]. Sua concepção partiu da observação de que a maioria dos algoritmos de ordenação clássicos foi projetada em ambientes acadêmicos, sem considerar os padrões estruturais frequentemente presentes em dados do mundo real [Auger et al. 2018]. O algoritmo fundamenta-se na combinação de dois algoritmos já consolidados na literatura: o *Merge Sort* e o *Insertion Sort*, aproveitando as propriedades de cada um conforme as características da entrada [Cormen et al. 2009, Sedgewick and Wayne 2011].

O princípio central do *Tim Sort* consiste em identificar subsequências naturalmente ordenadas no vetor de entrada, denominadas *runs*, e utilizá-las como unidades-base para o processo de fusão [Peters 2002]. Caso uma *run* identificada seja mais curta do que um tamanho mínimo predefinido, denominado *minrun*, o algoritmo a estende por meio do *Insertion Sort*, que apresenta desempenho eficiente para entradas pequenas ou parcialmente ordenadas [Auger et al. 2018]. O valor de *minrun* é calculado para garantir que o número de *runs* seja igual ou ligeiramente inferior a uma potência de dois, o que maximiza a eficiência da etapa de fusão posterior [Peters 2002].

Após a identificação e ordenação das *runs*, o *Tim Sort* gerencia sua fusão por meio de uma pilha (*stack*), mantendo invariantes sobre os tamanhos das *runs* no topo da estrutura [Auger et al. 2018]. Para três *runs* consecutivas no topo da pilha, denominadas $X$, $Y$ e $Z$, o algoritmo impõe as condições $|Z| > |Y| + |X|$ e $|Y| > |X|$, garantindo fusões balanceadas e limitando a profundidade da pilha a $O(\log n)$ [Auger et al. 2018]. Quando essas invariantes são violadas, fusões são disparadas até que sejam restabelecidas. Ao final, as *runs* remanescentes na pilha são fundidas sucessivamente para produzir o vetor ordenado.

Do ponto de vista da complexidade assintótica, o *Tim Sort* apresenta complexidade

de tempo $O(n \log n)$ no pior caso e $O(n)$ no melhor caso, quando a entrada já se encontra ordenada [Auger et al. 2018]. A prova formal da complexidade de pior caso $O(n \log n)$ foi estabelecida por [Auger et al. 2018], que também demonstrou que o tempo de execução é $O(n + n \log \rho)$, onde $\rho$ representa o número de *runs* na entrada, evidenciando a adaptabilidade do algoritmo a dados parcialmente ordenados. Essa análise também revelou um *bug* na implementação da versão Java do algoritmo, capaz de causar falha durante a execução [Auger et al. 2018].

Em relação à complexidade de espaço, o *Tim Sort* requer memória auxiliar de ordem $O(n)$ no pior caso, utilizada durante a etapa de fusão [Sedgewick and Wayne 2011]. Na prática, entretanto, o algoritmo opera com espaço significativamente menor, uma vez que apenas a menor das duas *runs* a serem fundidas é copiada para o vetor auxiliar [Peters 2002]. Assim como o *Merge Sort*, o *Tim Sort* é classificado como um algoritmo **estável** e **não in-place**, preservando a ordem relativa de elementos iguais ao longo de toda a execução [Cormen et al. 2009].

Em termos de adoção prática, o *Tim Sort* tornou-se o algoritmo de ordenação padrão em diversas plataformas e linguagens de programação amplamente utilizadas, incluindo Python (versões 2.3 a 3.10), Java SE 7, Android e GNU Octave [Auger et al. 2018, Peters 2002]. Essa ampla adoção reflete sua superioridade empírica sobre alternativas clássicas em cenários com dados reais, validando a abordagem híbrida e adaptativa como uma estratégia eficaz para ordenação de propósito geral [Sedgewick and Wayne 2011]. A Listagem 2 apresenta uma implementação em pseudocódigo do algoritmo.

**Listing 2. Implementação em Pseudocódigo do Tim Sort.**

```
FUNCAO calcularMinRun(n)
  r = 0
  ENQUANTO n >= 32 FACA
    r = r OU (n E 1)   // bit menos significativo
    n = n >> 1         // deslocamento a direita
  FIM ENQUANTO
  RETORNA n + r
FIM FUNCAO

PROCEDIMENTO timSort(v[], n)
  minRun = calcularMinRun(n)

  // Etapa 1: ordenar cada run com Insertion Sort
  PARA inicio = 0 ATE n-1 PASSO minRun FACA
    fim = MIN(inicio + minRun - 1, n - 1)
    insertionSort(v, inicio, fim)
  FIM PARA

  // Etapa 2: fundir runs sucessivamente
  tamanho = minRun
  ENQUANTO tamanho < n FACA
    PARA esq = 0 ATE n-1 PASSO 2 * tamanho FACA
      meio = MIN(esq + tamanho - 1, n - 1)
      dir  = MIN(esq + 2 * tamanho - 1, n - 1)
      SE meio < dir ENTAO
        merge(v, esq, meio, dir)  // mesmo merge do Merge Sort
      FIM SE
    FIM PARA
```

```
    tamanho = tamanho * 2
  FIM ENQUANTO
FIM PROCEDIMENTO

PROCEDIMENTO insertionSort(v[], esq, dir)
  PARA i = esq + 1 ATE dir FACA
    chave = v[i]
    j = i - 1
    ENQUANTO j >= esq E v[j] > chave FACA
      v[j + 1] = v[j]
      j = j - 1
    FIM ENQUANTO
    v[j + 1] = chave
  FIM PARA
FIM PROCEDIMENTO
```

## 3. Metodologia

Esta seção descreve o ambiente computacional utilizado nos experimentos, incluindo as bibliotecas empregadas na implementação dos algoritmos, as especificações de hardware e o sistema operacional adotado, visando garantir que os resultados possam ser reproduzidos. São apresentados ainda os métodos utilizados para coleta e processamento dos dados, assim como as ferramentas de medição e análise de desempenho. Por fim, são detalhados os critérios e procedimentos definidos para avaliar e comparar o desempenho e o comportamento dos algoritmos de ordenação frente a diferentes configurações de entrada.

### 3.1. Especificações Técnicas

Nesta subseção é apresentado a configuração da máquina utilizada para a execução dos testes, fornecendo informações essenciais para a reprodução dos resultados. Essas informações permitem a contextualização dos resultados obtidos, demonstrando as condições sob as quais os experimentos foram realizados. A Tabela 1 descreve as especificações da máquina.

**Tabela 1. Especificações Técnicas da Máquina Utilizada**

| Especificação | Descrição |
|---|---|
| Processador | AMD Ryzen 9 7950X |
| Memória RAM | 32,0 GB @ 6000 MT/s |
| Sistema Operacional | Windows 11 Pro |

### 3.2. Especificações Práticas

Nos experimentos foram desenvolvidas funções específicas em linguagem Python para a geração dos vetores de entrada, contemplando distribuições crescentes, decrescentes e aleatórias, de modo a controlar a ordenação inicial dos dados. Os vetores crescentes foram gerados como sequências inteiras contíguas por meio da função `range`, enquanto os decrescentes foram obtidos pela inversão dessa mesma sequência. Para os vetores aleatórios, utilizou-se o gerador `Random` da biblioteca padrão do Python, com semente fixa definida como `42 + tamanho`, garantindo a reprodutibilidade dos conjuntos de entrada entre execuções distintas. Essa padronização dos conjuntos de teste permitiu isolar

o impacto da organização dos dados sobre o comportamento dos algoritmos, conectando a geração dos vetores às análises de desempenho realizadas posteriormente.

Os cenários experimentais consideraram vetores com 100.000, 500.000 e 1.000.000 elementos, sendo cada tamanho avaliado sob as três configurações de ordenação inicial definidas. Essa combinação sistemática de tamanhos e distribuições assegurou uma cobertura abrangente dos casos analisados, estabelecendo uma base consistente para a comparação entre *Merge Sort* e *Tim Sort*.

A execução repetida de experimentos e o uso de médias estatísticas são práticas recomendadas na avaliação empírica de algoritmos, contribuindo para maior confiabilidade e reprodutibilidade dos resultados experimentais [McConnell 2008].

Para a medição do tempo de execução, foi utilizada a função `perf_counter` do módulo `time` da biblioteca padrão do Python, que fornece um contador de alta resolução e comportamento monotônico, adequado para medições de desempenho de curta duração. Essa abordagem permitiu a obtenção do tempo decorrido em segundos com alta precisão, conectando a medição temporal às demais análises realizadas durante a execução dos algoritmos.

Cada cenário foi executado 30 vezes para ambos os algoritmos, sem descarte de execuções iniciais, de modo que todas as 30 medições foram consideradas no cálculo da média aritmética. Os resultados foram consolidados diretamente em arquivos CSV, organizados por algoritmo no diretório de resultados, os quais serviram de base para a geração dos gráficos por meio de um *script* em Python utilizando a biblioteca `matplotlib.pyplot`. Observe a Tabela 2.

**Tabela 2. Critérios de Avaliação e Justificativas**

| Critério de Avaliação | Justificativa para a Escolha |
|---|---|
| Tempo Médio de Execução (s) | Permite avaliar o desempenho temporal dos algoritmos, sendo uma métrica fundamental para a comparação da eficiência prática sob diferentes tamanhos e configurações de entrada. |
| Complexidade Assintótica Observada | Permite analisar empiricamente a taxa de crescimento do custo computacional em função do tamanho da entrada, possibilitando a comparação entre comportamento teórico e prático. |

## 4. Resultados e Discussão

Nesta seção, são apresentados e analisados os dados obtidos nos testes de desempenho dos algoritmos de ordenação *Merge Sort* e *Tim Sort*, considerando diferentes cenários de ordenação em larga escala. Os dados são discutidos à luz das características teóricas de cada algoritmo, buscando relacionar os resultados práticos com o comportamento esperado das implementações avaliadas.

### 4.1. Tempo Médio de Execução

Os resultados apresentados na Figura 1 revelam um padrão notável no *Merge Sort*: os cenários crescente e decrescente apresentam tempos praticamente idênticos em todos os tamanhos testados, com diferença inferior a 0,3% para 100.000 elementos (0,103 s contra 0,103 s) e inferior a 0,1% para 1.000.000 elementos (1,182 s contra 1,183 s). Esse

comportamento é coerente com a natureza do algoritmo: por dividir o vetor recursivamente em metades e mesclar os resultados de forma sistemática, o *Merge Sort* realiza um volume de trabalho estruturalmente semelhante independentemente de os dados estarem ordenados crescente ou decrescentemente [Sedgewick and Wayne 2011].

O cenário aleatório consolida-se como o mais custoso, configurando-se como o pior caso prático nesta implementação. Para 1.000.000 de elementos, o tempo médio de 2,115 s representa aproximadamente 1,79 vezes o tempo do cenário crescente (1,182 s). Essa diferença decorre do comportamento da etapa de mesclagem: em vetores ordenados, uma das metades tende a ser esgotada rapidamente durante a comparação, reduzindo o número efetivo de comparações por nível de recursão. Em vetores aleatórios, as comparações se alternam com maior frequência entre as duas metades, elevando o custo total da operação [Knuth 1998].

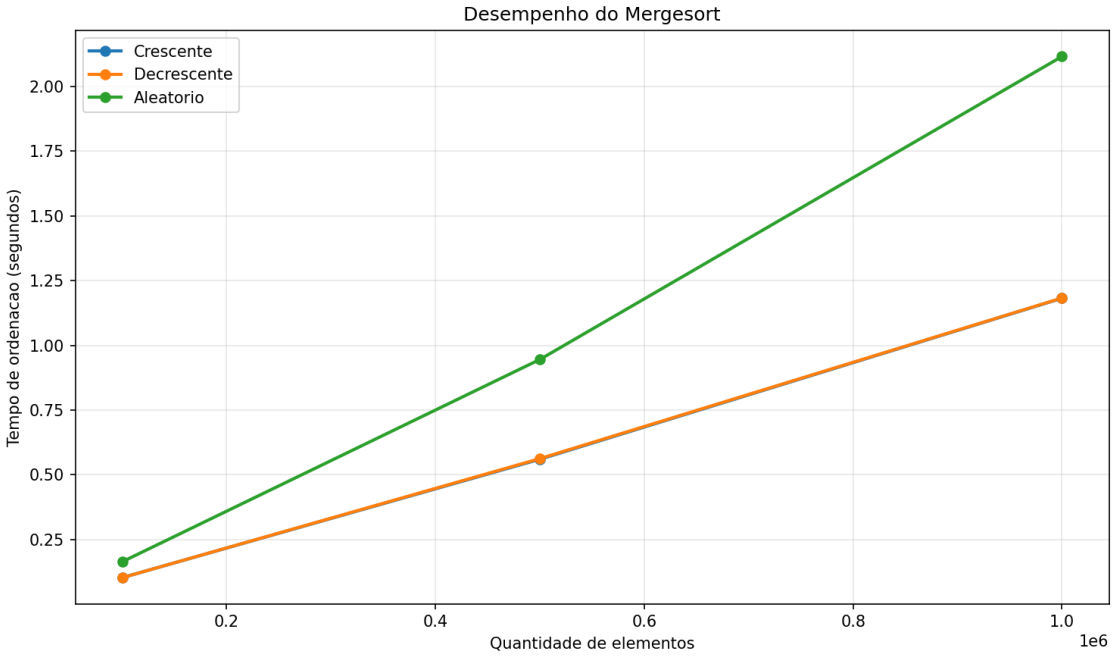


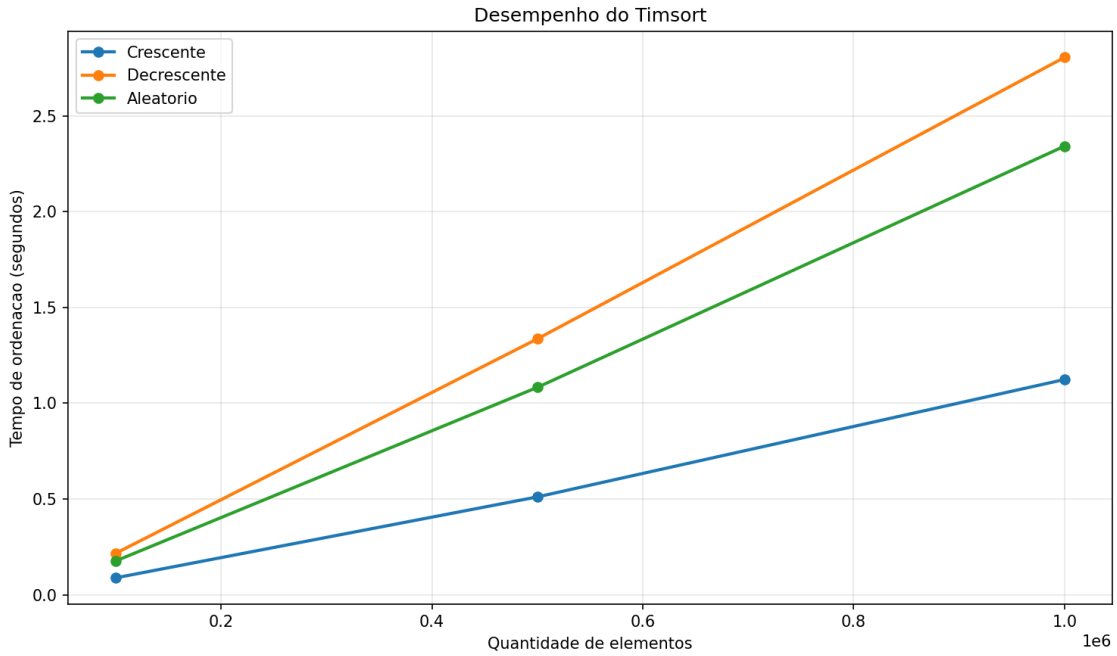

**Figura 1. Comparação de Tempo Médio de Execução do Merge Sort**

Os dados da Figura 2 expõem um comportamento distinto e contra-intuitivo no *Tim Sort*: o cenário decrescente torna-se o mais custoso, superando inclusive o aleatório em todos os tamanhos avaliados. Para 1.000.000 de elementos, o cenário decrescente atingiu 2,805 s, valor 2,50 vezes superior ao crescente (1,124 s) e 1,20 vezes superior ao aleatório (2,342 s). Essa inversão em relação ao comportamento esperado de um algoritmo adaptativo explica-se pela implementação da fase de pré-ordenação: a função de *insertion sort* aplicada a cada bloco de tamanho *minrun* exige o máximo de deslocamentos quando os elementos estão em ordem estritamente decrescente, pois cada novo elemento deve ser movido até o início do bloco já processado [**?**]. Com aproximadamente $n/minrun$ blocos sendo processados desta forma, o custo acumulado da fase de inserção degrada significativamente o desempenho global do algoritmo.

O cenário crescente representa o melhor caso do *Tim Sort*, com tempo de 1,124 s para 1.000.000 de elementos. Nesse cenário, o *insertion sort* confirma a ordem em cada bloco com apenas uma comparação por elemento, reduzindo a fase de inserção a custo

linear. A fase de mesclagem dos blocos já ordenados prossegue então com eficiência máxima.

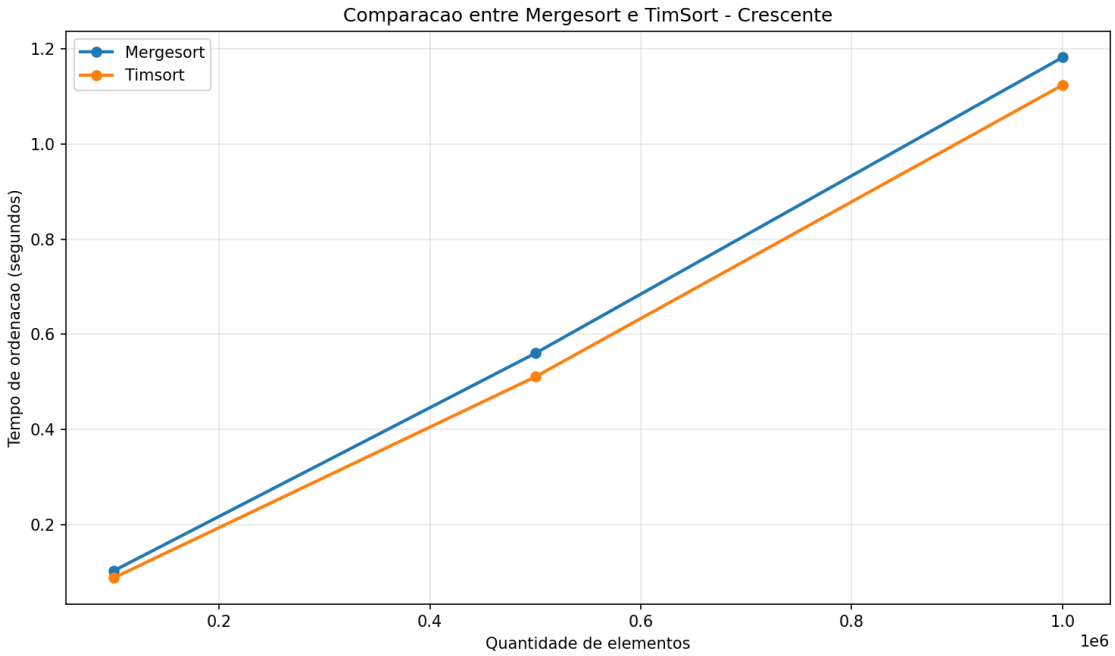
**Figura 2. Comparação de Tempo Médio de Execução do Tim Sort**




A análise comparativa entre os algoritmos, ilustrada nas Figuras 3, 4 e 5, evidencia que o desempenho relativo varia substancialmente conforme a distribuição inicial dos dados.

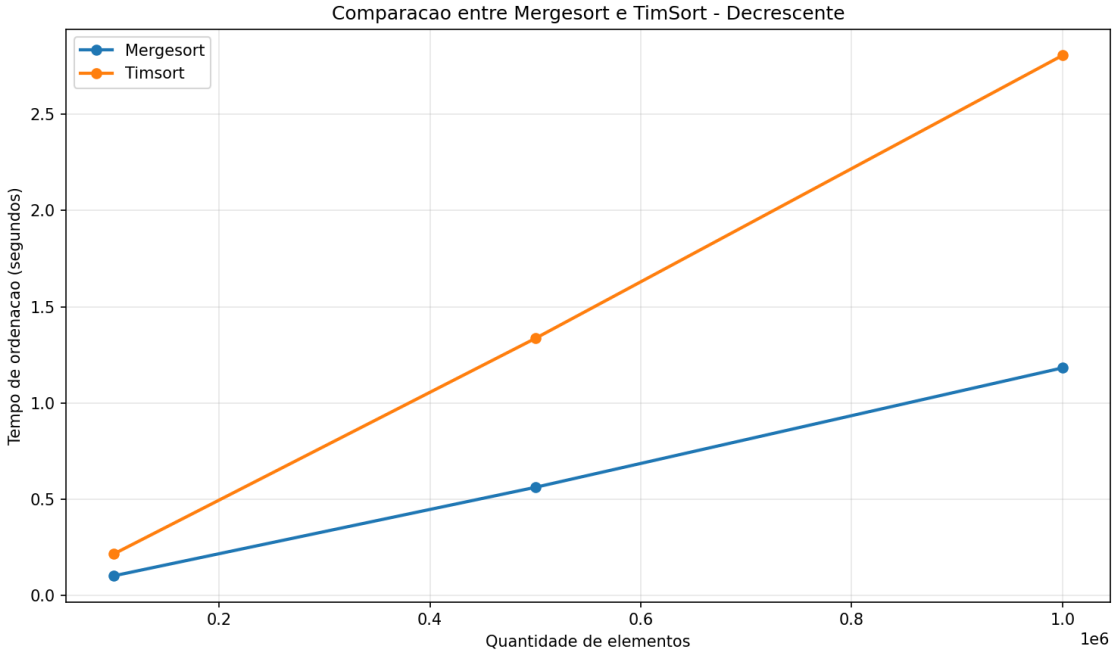
No cenário crescente (Figura 3), o *Tim Sort* apresenta leve vantagem sobre o *Merge Sort* em todos os tamanhos, com tempo inferior em aproximadamente 5% para 1.000.000 de elementos (1,124 s contra 1,182 s). Esse ganho marginal reflete a eficiência da fase de inserção em dados já ordenados, mas não representa uma vantagem expressiva dado o nível de sobreposição entre as curvas.

**Figura 3. Comparação de Tempo Médio de Execução (Cenário Crescente)**



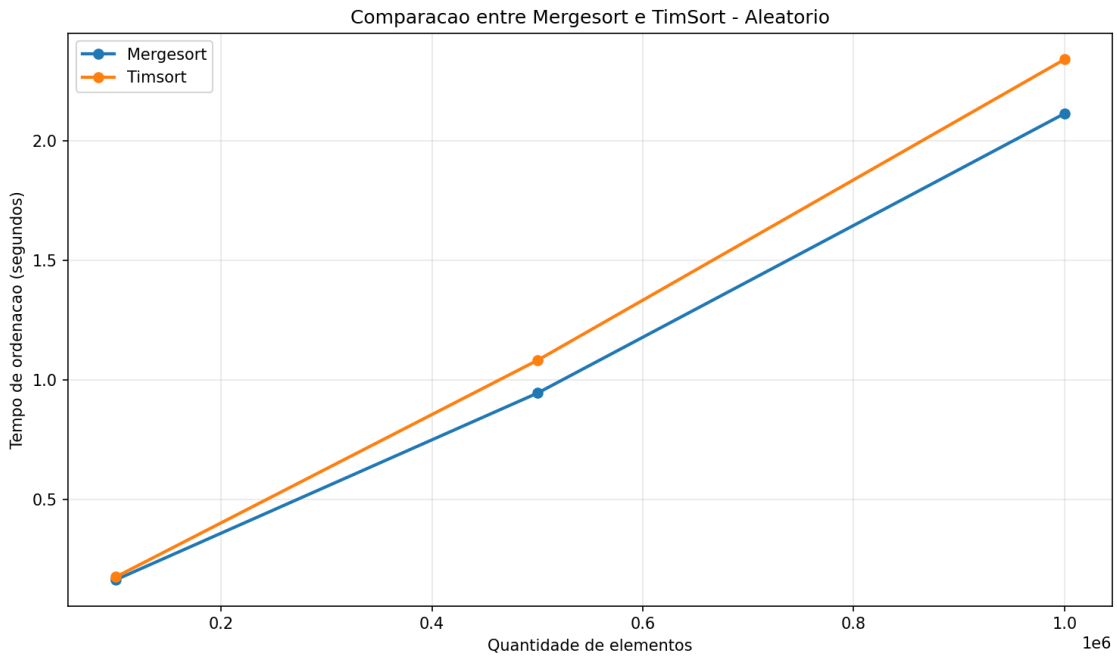

No cenário decrescente (Figura 4), a diferença torna-se acentuada e favorável ao *Merge Sort*. Para 1.000.000 de elementos, o *Merge Sort* completa a ordenação em 1,183 s, enquanto o *Tim Sort* demanda 2,805 s — uma diferença de aproximadamente 2,37 vezes. Esse resultado demonstra que a sobrecarga da fase de *insertion sort* sobre blocos inversamente ordenados supera o benefício da estratégia adaptativa do *Tim Sort*, tornando-o inferior ao *Merge Sort* clássico nesse cenário específico.

**Figura 4. Comparação de Tempo Médio de Execução (Cenário Decrescente)**




No cenário aleatório (Figura 5), o *Merge Sort* mantém vantagem sobre o *Tim Sort* para todos os tamanhos avaliados, atingindo 2,115 s contra 2,342 s para 1.000.000 de elementos — uma diferença de aproximadamente 11%. Sem estrutura prévia nos dados para ser explorada pela fase adaptativa, o *Tim Sort* não consegue amortizar sua maior complexidade de implementação, o que favorece a abordagem direta e uniforme do *Merge Sort*.

**Figura 5. Comparação de Tempo Médio de Execução (Cenário Aleatório)**




### 4.2. Complexidade Assintótica Observada

A análise empírica da taxa de crescimento dos tempos de execução, obtida a partir da razão entre os tempos medidos para os tamanhos extremos (100.000 e 1.000.000 elementos, razão de escala de 10 vezes), permite verificar a aderência dos resultados à complexidade assintótica teórica $O(n \log n)$. Para essa escala, o modelo teórico prevê um fator de crescimento de aproximadamente $10 \times \frac{\log_2 1.000.000}{\log_2 100.000} \approx 12{,}0$ vezes.

Para o *Merge Sort*, os fatores observados foram de 11,5 vezes no cenário crescente e 12,9 vezes no aleatório, ambos consistentes com a previsão teórica. A regularidade entre os cenários crescente e decrescente confirma que o algoritmo mantém complexidade $\Theta(n \log n)$ independentemente da ordenação inicial, conforme esperado por sua estrutura de divisão e conquista.

Para o *Tim Sort*, os fatores observados foram de 12,8 vezes no cenário crescente e 13,0 vezes no decrescente. O cenário decrescente, embora apresente o maior tempo absoluto, exibe crescimento proporcional compatível com $O(n \log n)$, indicando que a degradação causada pela fase de *insertion sort* representa uma constante multiplicativa elevada, e não uma piora na ordem de crescimento assintótico. O cenário aleatório apresentou fator de 13,3 vezes, também dentro do intervalo esperado para $O(n \log n)$.

Em síntese, ambos os algoritmos confirmam empiricamente a complexidade assintótica $O(n \log n)$ em todos os cenários testados. A distinção entre eles reside nos coeficientes práticos: o *Tim Sort* apresenta coeficiente favorável apenas no cenário crescente, enquanto o *Merge Sort* demonstra maior uniformidade entre os cenários, consolidando-se como a implementação de menor custo médio no contexto das avaliações realizadas.

## 5. Conclusão

Os resultados experimentais confirmaram que ambos os algoritmos respeitam a complexidade assintótica $O(n \log n)$ em todos os cenários avaliados, com fatores de crescimento

empíricos próximos ao valor teórico esperado de 12,0 vezes para uma escala de $10\times$. Contudo, os coeficientes práticos revelaram diferenças relevantes entre as implementações.

O *MergeSort* demonstrou comportamento uniforme e previsível independentemente da ordenação inicial dos dados, com tempos praticamente idênticos nos cenários crescente e decrescente, consolidando-se como a implementação de menor custo médio no contexto avaliado. O *TimSort*, por sua vez, apresentou desempenho superior apenas no cenário crescente, onde sua fase adaptativa de *Insertion Sort* opera com custo linear, reduzindo o tempo total em aproximadamente 5% em relação ao *MergeSort*. Nos cenários decrescente e aleatório, o *TimSort* mostrou-se inferior, com destaque para o cenário decrescente, no qual o custo acumulado da fase de inserção sobre blocos inversamente ordenados elevou o tempo de execução a 2,37 vezes o do *MergeSort*.

Conclui-se que a adaptatividade do *TimSort* constitui uma vantagem efetiva apenas quando os dados apresentam estrutura crescente parcial ou total, cenário típico em aplicações do mundo real para o qual o algoritmo foi originalmente concebido. Em contextos onde a distribuição dos dados é desconhecida ou predominantemente desordenada, o *MergeSort* oferece garantias mais robustas de desempenho. A escolha entre os algoritmos deve, portanto, considerar o perfil esperado da entrada, sendo inadequado eleger um deles como universalmente superior sem essa contextualização.

Esses resultados reforçam a importância de avaliar algoritmos não apenas do ponto de vista teórico, mas também experimental, prática amplamente defendida na literatura clássica de análise de desempenho de software e algoritmos [Weiss 2014].

## Referências


Aho, A. V., Hopcroft, J. E., and Ullman, J. D. (1974). *The Design and Analysis of Computer Algorithms*. Addison-Wesley.

Auger, N., Jugé, V., Nicaud, C., and Pivoteau, C. (2018). On the worst-case complexity of TimSort. In *26th Annual European Symposium on Algorithms (ESA 2018)*, volume 112 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 4:1–4:13. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

Bentley, J. L. and McIlroy, M. D. (1993). Engineering a sort function. *Software: Practice and Experience*, 23(11):1249–1265.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3 edition.

Gonnet, G. H. and Baeza-Yates, R. (1991). *Handbook of Algorithms and Data Structures*. Addison-Wesley.

Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2 edition.

McConnell, S. (2008). *Code Complete*. Microsoft Press, 2 edition.

McIlroy, P. M. (1993). Optimistic sorting and information theoretic complexity. pages 467–474.

Peters, T. (2002). TimSort description. CPython source repository. Acessado em: abr. 2025.

Sedgewick, R. and Wayne, K. (2011). *Algorithms*. Addison-Wesley Professional, Upper Saddle River, NJ, 4 edition.

Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in Java*. Pearson, 3 edition.

Wiredu, J. K., Akobre, S., Aabaah, I., and Wumpini, U. A. (2025). A unified framework for theoretical and experimental evaluation of classical and modern sorting algorithms in real-time systems. *Journal of Information Systems and Informatics*, 7(4):3143–3185.